Téma 7: Práce se soubory (Čtení a Zápis)

Teoretický základ

Aby výsledky našeho programu nezmizely po jeho vypnutí, ukládáme je na pevný disk.

- Pro práci se souborem se v Pythonu používá funkce `open(nazev, mod)`.
- Nejčastější pracovní módy (modes): `'r'` (read, výchozí), `'w'` (write, vše přepíše), `'a'` (append, přidává na konec).
- Nikdy nesmíme zapomenout soubor zavřít `f.close()`! Z toho důvodu je standardem v Pythonu používat tzv. Context Manager blok `with open(...) as f:`, který provede bezpečné uzavření souboru automaticky i při chybě v programu.

Základní koncept k zamyšlení: Limit paměti

Při čtení dat ze souboru máme tři hlavní metody: `read()` načte celý obsah do jednoho řetězce, `readlines()` načte řádky do pole. Pokud bychom ale načítali logovací soubor o velikosti 10 GB, volání `read()` by zcela zahltilo paměť RAM a program by spadl. U velkých souborů se vždy využívá třetí způsob - čtení pomocí `for radek in soubor:`, který zpracovává data paměťově bezpečně řádek po řádku.

Úkol 1 (Lehká obtížnost)

Zadání úkolu: Napište program, který vytvoří úplně nový textový soubor (s názvem `moje_jmeno.txt`) a zapíše do něj vaše křestní jméno. Použijte nejbezpečnější variantu zápisu, tedy tzv. context manager (strukturu `with open...`).

Řešení úkolu 1

```
# Parametr 'w' (write) soubor založí nebo kompletně promaže a přepíše.
# Blok with se automaticky a bezpečně postará o zavření (soubor.close())
# a to i v případě, že by program uprostřed zápisu havaroval.

with open("moje_jmeno.txt", "w", encoding="utf-8") as soubor:
    soubor.write("Jan Novák")
```

Úkol 2 (Střední obtížnost)

Zadání úkolu: Otevřete soubor vytvořený v předchozím úkolu pro čtení. Přečtěte jeho obsah a vypište tento obsah na obrazovku VELKÝMI PÍSMENY (využitím stringové metody `.upper()`).

Řešení úkolu 2

```
# Parametr 'r' (read) slouží k bezpečnému otevření pouze pro čtení
with open("moje_jmeno.txt", "r", encoding="utf-8") as soubor:

    # Načteme celý obsah souboru do jedné proměnné naráz
    obsah = soubor.read()

    # Textový řetězec změníme na velká písmena a vypíšeme
    print(obsah.upper())
```

Úkol 3 (Těžká obtížnost)

Zadání úkolu: Představte si textový log, který se jmenuje `server_log.txt`. Na každém řádku je událost (např. *"Info: Start"*). Napište filtr, který soubor postupně řádek po řádku přečte a do nového souboru `chyby_log.txt` zkopíruje jenom ty řádky, které začínají slovem *"Error"*.

Řešení úkolu 3

```
# Pro tento úkol musíme otevřít dva soubory současně vedle sebe
with open("server_log.txt", "r", encoding="utf-8") as vstup:
    with open("chyby_log.txt", "w", encoding="utf-8") as vystup:

        # Tato iterace zpracovává vždycky jen jeden řádek (šetří paměť RAM)
        for radek in vstup:

            # strip() vyčistí skryté znaky jako \n. startswith ověřuje
začátek
            if radek.strip().startswith("Error"):
                vystup.write(radek)
```

Úkol 4 (Lehká obtížnost - dodatek)

Zadání úkolu: Máme na disku soubor (třeba i ten z úkolu 1). Pokus byste u něj použili běžný mód pro zápis `"w"`, celý starý obsah by se zničil. Napište proto úsek kódu s nejbezpečnějším `with open` a otevřete soubor v módu **Append** (přidávání). Připište na konec souboru novou větu.

Řešení úkolu 4

```
# Mód 'a' (Append) umístí pomyslný kurzor na úplný konec existujícího
# souboru a cokoliv nového tam bezpečně připojí, aniž by smazal historii.

with open("moje jmeno.txt", "a", encoding="utf-8") as soubor:
    # Všimněte si úvodního \n - zajistí přeskok na nový řádek před zápisem
    soubor.write("\nTato věta byla bezpečně připsána až na konec.")
```

Úkol 5 (Střední obtížnost - dodatek)

Zadání úkolu: Napište program, který otevře libovolný textový soubor obsahující celistvý delší text (např. `clanek.txt`) pro čtení. Úkolem programu je vyhodnotit a vypsat, kolik se v daném souboru nachází celkem slov.

Řešení úkolu 5

```
# Předpokládáme, že existuje nějaký soubor s textem
with open("clanek.txt", "r", encoding="utf-8") as soubor:
    cely_text = soubor.read()

    # Metoda .split() bez parametrů automaticky naseká řetězec
    # v místě jakýchkoliv mezer nebo odřádkování do samostatných prvků v
poli
    vsechna_slova = cely_text.split()

    # Funkce len() nám poví, kolik takto oddělených kusů v poli je
    pocet_slov = len(vsechna_slova)

print(f"Tento soubor obsahuje celkem {pocet_slov} slov.")
```

Úkol 6 (Těžká obtížnost - dodatek)

Zadání úkolu: Pokud programu přikážete načíst (mód `"r"`) soubor, který neexistuje, celý software havaruje v červených chybách. Použijte zachytávací bloky `try / except` k tomu, abyste odchytili specifickou výjimku při neexistenci souboru a uživateli suše vypsali: *"Soubor nebyl nalezen."*

Řešení úkolu 6

```
# Zkusíme provést nebezpečnou operaci
try:
    with open("zahadny soubor.txt", "r", encoding="utf-8") as soubor:
        obsah = soubor.read()
        print(obsah)

# Tento blok čeká, dokud Python z try-bloku nevyhodí konkrétně
# chybu FileNotFoundError. Pokud ano, odchytí ji a zabrání pádu aplikace.
except FileNotFoundError:
    print("Kritická chyba: Soubor nebyl nalezen a nelze jej načíst.")
```